CMPE 492

Automated Game Testing and Difficulty Assessment

Alperen AKYOL

Advisor:

Atay Özgövde

March 30, 2026

TABLE OF CONTENTS

1. INTRODUCTION

1.1. Broad Impact

The implementation of a curiosity-driven reinforcement learning (RL) framework for automated testing carries significant implications for the gaming industry, particularly within the fast-paced mobile development landscape.

1.1.1. Operational Efficiency in Rapid Development Cycles

Modern mobile games operate as live services with frequent content updates, balance adjustments, and time-limited events that continuously modify game logic. Manual quality assurance (QA) processes often struggle to keep pace with these rapid changes. By leveraging curiosity-driven exploration, the proposed system can autonomously discover edge cases and logic errors—such as soft-lock scenarios caused by minor level modifications—that traditional scripted bots or fatigued human testers may overlook.

1.1.2. Enhancing Accessibility and Player Retention

Mobile games serve a diverse and heterogeneous player base with varying skill levels. By simulating distinct player profiles (e.g., "casual," "average," and "expert"), this framework enables developers to quantitatively evaluate difficulty consistency across different segments of players. This contributes to maintaining a balanced experience, ensuring that games remain engaging for experienced players without becoming discouragingly difficult for newcomers, ultimately improving player retention and satisfaction.

1.1.3. Democratization for Indie and Mid-Sized Studios

Comprehensive human playtesting is resource-intensive and often economically prohibitive for smaller studios. This project introduces a scalable and cost-effective testing assistant capable of simulating extensive gameplay scenarios within a short time frame. By automating repetitive and exhaustive exploration tasks, the framework allows smaller teams to achieve levels of polish and robustness comparable to those of large-scale (AAA) studios.

1.1.4. Bridging the Gap Between AI and Game Design

A major barrier to the adoption of reinforcement learning in game testing is the lack of interpretability of learned behaviors. This framework addresses this issue by transforming raw agent interactions into actionable insights, such as coverage heatmaps and reproducible failure cases. As a result, designers can make informed, data-driven decisions regarding level design and difficulty balancing without requiring in-depth expertise in reinforcement learning techniques.

1.2. Ethical Considerations

As analytical responsibilities are increasingly delegated to neural networks, several ethical considerations must be addressed to ensure that the proposed framework remains beneficial to the industry rather than disruptive.

1.2.1. Augmentation vs. Displacement

A primary ethical concern is the potential displacement of entry-level Quality Assurance (QA) personnel. This framework is designed as an augmentative tool that handles mathematically intensive and repetitive state-space exploration. By automating the detection of logic errors and soft-lock scenarios, it enables human testers to focus on subjective aspects such as gameplay experience, narrative coherence, and overall enjoyment—areas that remain beyond the capabilities of current reinforcement

learning agents.

1.2.2. Representation in Player Profiling

The modeling of player profiles (e.g., “casual,” “average,” and “expert”) must be approached carefully to avoid algorithmic bias. If these profiles are constructed based on limited or unrepresentative data, the resulting difficulty balancing may unintentionally exclude certain player groups or accessibility needs. It is therefore essential that parameters such as perception noise and action precision reflect a diverse spectrum of human abilities and play styles.

1.2.3. Integrity of Game Balance

Reinforcement learning agents may discover exploits that are not feasible for human players, such as frame-perfect inputs or unintended mechanical interactions. Relying on such behaviors for difficulty evaluation may result in misleading conclusions, where levels are considered solvable only through unrealistic strategies. The system must therefore distinguish between intended solutions and unintended exploits to preserve fair and meaningful game balance.

1.2.4. Responsible Difficulty Engineering

While the framework provides detailed insights into difficulty progression and potential bottlenecks, its application remains the responsibility of developers. These insights should be used to enhance player experience by ensuring a fair and engaging learning curve. Misuse of such data to introduce artificial difficulty spikes or manipulative progression systems—such as those designed to incentivize microtransactions—raises ethical concerns and should be avoided.

2. PROJECT DEFINITION AND PLANNING

2.1. Project Definition

The primary objective of this project is to develop a modular, automated testing and difficulty assessment framework for level-based puzzle games using curiosity-driven Reinforcement Learning (RL). The framework is based on a multi-layer architecture consisting of a Learning Layer for agent training, a Coordination Layer for managing exploration, and an Analysis Layer for generating developer-oriented insights.

2.1.1. Problem Statement

Modern mobile game development, particularly in live-service (Live-Ops) environments, requires rapid and continuous content iteration. Manual quality assurance (QA) is constrained by human fatigue and limited coverage of large state spaces. Additionally, scripted bots often lack the flexibility required to uncover non-intuitive edge cases or soft-lock scenarios that emerge from complex player interactions.

2.1.2. Proposed Solution and Scope

This project addresses these limitations by implementing agents enhanced with an Intrinsic Curiosity Module (ICM), which incentivizes exploration of novel states. To ensure computational efficiency, the game environments are implemented as Python-based command-line interface (CLI) applications, avoiding the overhead of traditional game engines.

The primary test environment is *Tile Blast*, selected due to its complex match-based mechanics and tile-grid structure. Compared to simpler path-planning puzzles, this environment provides a richer and more realistic testbed for evaluating exploration strategies and difficulty stability.

2.2. Project Planning

2.2.1. Project Time and Resource Estimation

The project is divided into three main phases aligned with the academic timeline:

- **Phase 1: Environment Development and Foundation (February – April 15)**
  - Completion of literature review and methodology definition
  - Development of the Python-based Tile Blast environment
- **Phase 2: Core RL Implementation (April 16 – May 25)**
  - Implementation of Proximal Policy Optimization (PPO)
  - Integration of the Intrinsic Curiosity Module (ICM)
  - Development of the Coordination Layer for parallel agents
- **Phase 3: Analysis and Evaluation (May 26 – June 11)**
  - Data collection for different player profiles
  - Generation of coverage heatmaps and difficulty metrics
  - Identification and logging of failure cases

Resources:

- **Hardware:** MSI Katana 15 workstation for training and simulation
- **Software:** Python 3.x, Gymnasium, Stable Baselines3

2.2.2. Success Criteria

The success of the framework will be evaluated using the following criteria:

- **State-Space Coverage:** High exploration coverage compared to a random baseline (currently validated on Worm Escape; Tile Blast results pending).
- **Edge-Case Identification:** Autonomous detection of known soft-lock scenarios
- **Analytical Utility:** Generation of consistent and interpretable difficulty metrics

2.2.3. Risk Analysis

- **Sparse Rewards:** Agents may struggle to learn in complex environments *Mitigation:* Adjust intrinsic reward weights to encourage exploration
- **Reward Hacking:** Agents may exploit curiosity signals without meaningful progress *Mitigation:* Refine observation space and apply reward constraints
- **Computational Complexity:** Large state space due to match-based mechanics *Mitigation:* Use optimized CLI-based simulation for faster execution

2.2.4. Team Work

This project is conducted individually by Alperen Akyol under the supervision of Professor Atay Özgövde.

3. RELATED WORK

Research on Reinforcement Learning (RL) for automated game testing has gained significant attention as modern games have become increasingly complex and dynamic. Traditional scripted testing methods often fail to capture emergent behaviors and rare edge cases, motivating the use of learning-based agents capable of adaptive exploration.

3.1. Curiosity-Driven Reinforcement Learning

A key challenge in automated game testing is the sparsity of reward signals, particularly in puzzle and logic-driven environments where meaningful outcomes occur only after long action sequences. To address this, curiosity-driven reinforcement learning introduces intrinsic rewards that encourage exploration of novel or unpredictable states.

Ferdous et al. [1] propose a curiosity-driven RL framework for 3D game testing, where agents are rewarded for encountering unfamiliar states. Their results show that such agents can discover hidden areas, rare interactions, and logic errors that are typically missed by scripted approaches. This work demonstrates that intrinsic motivation can significantly improve exploration coverage without requiring handcrafted reward functions.

3.2. Multi-Agent Exploration for State-Space Coverage

While curiosity-driven agents improve exploration, single-agent approaches often suffer from redundancy and limited coverage. To overcome this limitation, multi-agent reinforcement learning (MARL) has been introduced to enable parallel exploration.

Ferdous et al. [2] extend their earlier work by proposing a coordinated multi-agent framework in which multiple agents explore the same environment simultaneously. A lightweight coordination mechanism reduces overlap between agents, increasing overall

efficiency and state-space coverage. Their findings show that multi-agent exploration accelerates the discovery of complex, multi-step issues that are difficult to uncover with a single agent.

3.3. Industrial Applications of RL in Game Testing

The practical applicability of RL in game development has been demonstrated by Bergdahl et al. [3], who integrated deep reinforcement learning into the testing pipelines at Electronic Arts (EA). Their system enables agents to perform navigation, interaction testing, and difficulty assessment within real game environments.

A key contribution of this work is the emphasis on interpretability and integration. Rather than replacing human testers, RL agents act as scalable assistants that generate reproducible gameplay traces, coverage statistics, and behavioral insights. This highlights the importance of producing developer-facing artifacts, not just optimizing agent performance.

3.4. Diversity-Driven Exploration via Evolutionary Methods

Beyond reinforcement learning, evolutionary approaches have also been applied to automated game testing. Zheng et al. [4] introduce the Wuji framework, which combines deep RL with evolutionary algorithms to maintain a diverse population of agents.

By promoting behavioral diversity, Wuji enables broader exploration of the environment and improves the detection of rare bugs and unexpected interactions. This approach is particularly effective in complex environments with sparse or incomplete reward signals, where traditional RL methods may struggle.

3.5. Summary and Research Gap

Existing work demonstrates that curiosity-driven exploration, multi-agent coordination, and diversity-promoting mechanisms are effective strategies for improving automated game testing. However, most prior studies focus on high-dimensional 3D environments or full-scale game engines, which introduce significant computational overhead and limit scalability.

In contrast, this project focuses on lightweight, high-speed simulation environments implemented as Python-based command-line interfaces. This design enables efficient state-space exploration while maintaining sufficient complexity for evaluating difficulty and uncovering edge cases. Furthermore, the proposed framework emphasizes the generation of interpretable outputs—such as coverage metrics, difficulty curves, and reproducible failure traces—bridging the gap between RL research and practical game design workflows.

4. METHODOLOGY

This chapter details the design and implementation of a reinforcement learning-based framework for automated game testing and difficulty assessment. The system is built to be modular, engine-agnostic, and optimized for high-speed simulation via a Python command-line interface (CLI).

4.1. Overview of the Framework

The proposed framework is structured around three interconnected layers: the **Learning Layer**, the **Coordination Layer**, and the **Analysis Layer**. This architecture ensures that exploration logic is separated from environmental physics and final data processing.

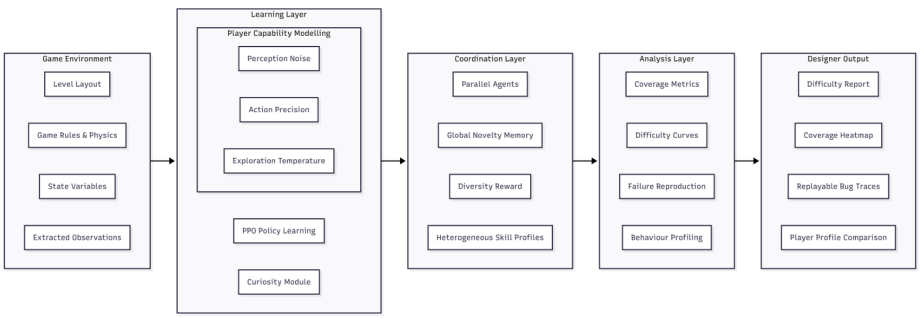

Figure 4.1. Overview of the proposed framework

The process begins by extracting structured observations from the game environment, which are then processed by a swarm of curiosity-driven agents to generate behavioral trajectories.

4.2. Game Environment and Modeling

To achieve the simulation throughput necessary for exhaustive state-space exploration, the framework utilizes a custom Python-based CLI rather than a traditional graphical engine.

4.2.1. Transition from Wiggle Escape to Tile Blast

Initial validation was conducted on *Wiggle Escape* using a directed dependency graph solver. The project then shifted focus to *Tile Blast* to address more complex, non-deterministic state-spaces. Unlike path-planning puzzles, *Tile Blast* introduces a 3D-stacked Mahjong-style board where the primary challenge is resource management.

4.2.2. State Representation and Buffer Logic

The "Extracted Observations" block in Figure 4.1 encompasses two primary data structures:

- **Board State:** A multidimensional representation of the tile grid, including tile types, layering/stacking order, and visibility.
- **Buffer (Tray) State:** A 7-slot resource buffer. The agent must select tiles that form triplets to clear them from the buffer.
- **Soft-lock Condition:** If the buffer reaches 7 tiles without forming a triplet, the episode terminates in failure, regardless of remaining matches on the board.

4.3. Learning Layer: Curiosity and Capability

The Learning Layer is responsible for training agents using *Proximal Policy Optimization (PPO)*.

4.3.1. Intrinsic Curiosity Module (ICM)

To navigate the sparse rewards of *Tile Blast*—where the goal is only reached after dozens of correct moves—agents utilize an intrinsic curiosity signal. The total reward R_t is defined as:

$$R_t = r_e + \eta \cdot r_i$$

where:

- r_e is the extrinsic game score
- r_i is the intrinsic reward derived from state-prediction errors
- η is a scaling factor for intrinsic motivation

This reward structure encourages agents to experiment with "buffer-filling" moves that might uncover critical tiles.

4.3.2. Player Capability Modeling

To evaluate game difficulty for a diverse audience, agents are modified across three dimensions:

- **Perception Noise:** Simulates "Casual" players by masking parts of the board or adding noise to tile identification.
- **Action Precision:** Models "Average" players through stochastic action selection (introducing randomness).
- **Exploration Temperature:** Adjusts the agent's "enthusiasm" for novelty to mimic different playstyles, from cautious to highly exploratory.

4.4. Coordination Layer: Parallel Swarm Exploration

Running a single agent is insufficient for discovering rare edge cases. The Coordination Layer manages multiple explorers operating in parallel:

- **Global Novelty Memory:** A shared database tracks which grid configurations and buffer states have already been visited.
- **Diversity Reward:** Agents are guided away from redundant trajectories, ensuring the swarm collectively covers the state-space efficiently.

4.5. Analysis Layer and Designer Output

The final layer converts raw trajectories into interpretable artifacts for game designers:

- **Coverage Metrics:** Measures the percentage of total game states explored.
- **Difficulty Curves:** Analyzes success rates and move counts across different player profiles to identify stability issues.
- **Reproducible Failure Traces:** Records the exact sequence of moves (the "replayable trace") that led to buffer-related soft-locks.

5. REQUIREMENTS SPECIFICATION

This chapter defines the functional and non-functional requirements of the curiosity-driven testing framework. The primary focus is on ensuring the system can autonomously navigate the Mahjong-like mechanics of *Tile Blast* while providing actionable data to game designers.

5.1. Use Case Diagrams

The system's interactions are defined by three primary actors: the **Game Designer**, the **RL Agent**, and the **System Coordinator**.

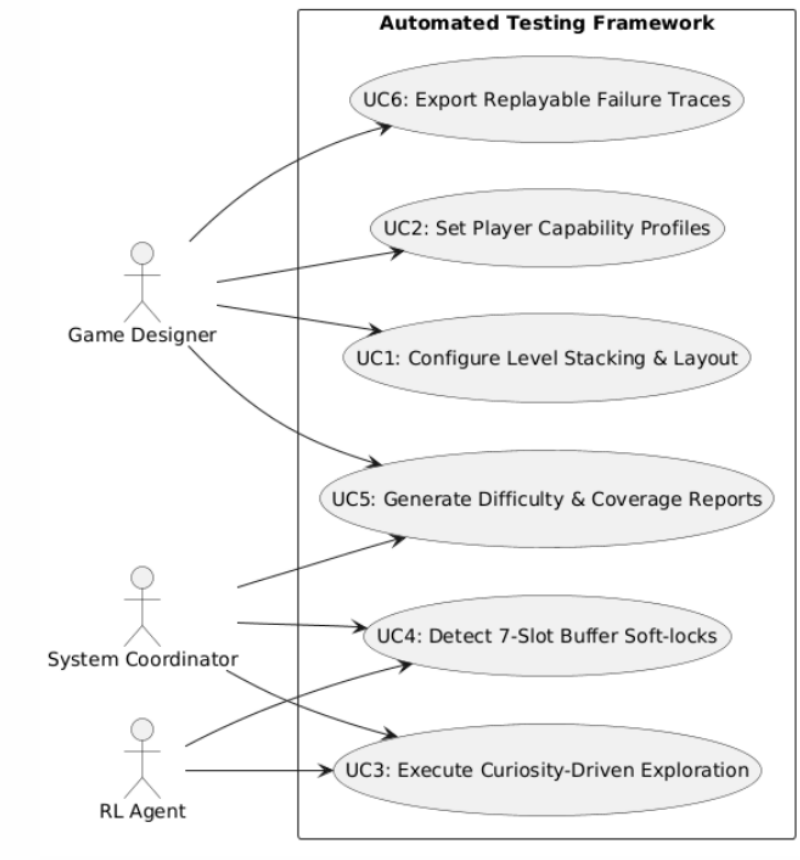

Figure 5.1. Use case diagram of the curiosity-driven game testing framework

5.1.1. Actor Descriptions

- **Game Designer:** Responsible for level configuration and interpreting analytical outputs such as difficulty curves and coverage metrics.
- **RL Agent:** The primary autonomous actor that interacts with the game environment to maximize exploration and discover failures.
- **System Coordinator:** Manages the multi-agent swarm, ensuring that the global novelty memory is updated to prevent redundant exploration.

5.2. Functional Requirements

The framework must fulfill the following technical capabilities to ensure a successful automated testing cycle:

- **FR1 – State Extraction:** The system shall extract a multidimensional state representation from the Python CLI, including tile types, 3D stacking order, and current visibility.
- **FR2 – Buffer Management:** The system shall maintain a buffer with a strict capacity of 7 slots.
- **FR3 – Match Logic Execution:** The system shall automatically clear triplets from the buffer and update the board state accordingly.
- **FR4 – Termination Conditions:** The system shall terminate an episode as a "Failure" if the 7-slot buffer reaches capacity without a triplet being formed (Soft-lock).
- **FR5 – Intrinsic Reward Calculation:** The learning layer shall calculate an intrinsic reward based on the prediction error of the next-state transition to encourage novelty.
- **FR6 – Multi-Agent Synchronization:** The coordinator shall synchronize the trajectories of parallel agents to ensure diverse state-space coverage.
- **FR7 – Artifact Generation:** The analysis layer shall generate reproducible failure traces and difficulty curves for designer review.

5.3. Non-Functional Requirements

- **NFR1 – Simulation Throughput:** The environment must be implemented as a lightweight Python CLI to allow for high-frequency state transitions without graphical overhead.
- **NFR2 – Reproducibility:** Every failure trace must include the initial seed and action sequence to allow for 100% deterministic replay by the designer.
- **NFR3 – Scalability:** The framework shall support the addition of new puzzle mechanics (e.g., special tiles or power-ups) with minimal modification to the core RL pipeline.

6. DESIGN

This chapter details the architectural and logical design of the curiosity-driven testing framework. The design emphasizes modularity, ensuring that the high-throughput Python CLI can interface seamlessly with the Reinforcement Learning (RL) components.

6.1. Information Structure

The information structure defines the static data entities and their logical relationships within the game environment. For the *Tile Blast* implementation, the model must accurately represent 3D tile stacking, visibility, and the temporal state of the match buffer.

6.1.1. Data Entities

- **Tile:** The fundamental unit of the game, characterized by its *TypeID*, *LayerDepth*, and *Position*. A tile's status is dynamically updated based on whether it is obscured by other tiles, active in the buffer, or cleared from the board.
- **Board:** A collection of tiles organized in a 3D coordinate system. It maintains the "blocking" logic, ensuring a tile is only interactive if no other tile partially overlaps it at a higher depth.
- **Buffer (Tray):** A critical resource entity with a fixed capacity of 7 slots. It tracks the sequence of selected tiles and triggers the "Clear" logic when a triplet of the same *TypeID* is formed.
- **Agent Observation:** A derived data structure that flattens the board representation and current buffer occupancy into a multidimensional tensor suitable for the PPO policy network.

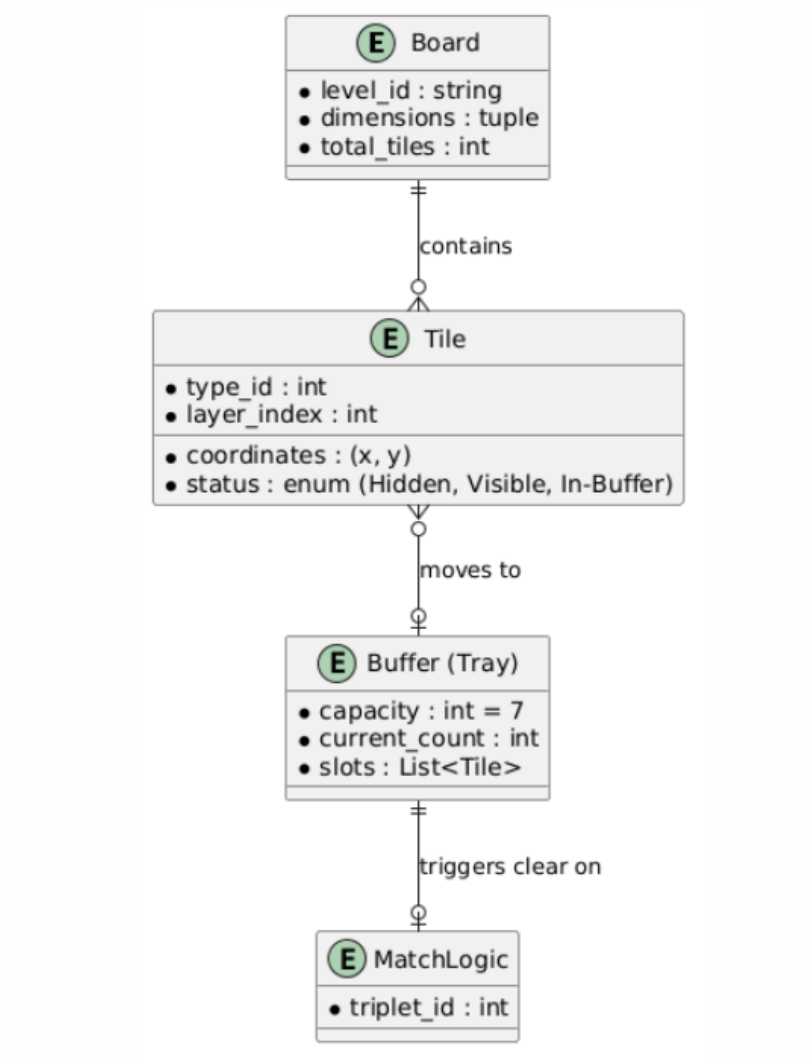

Figure 6.1. Entity Relationship Diagram (ERD) of the Tile Blast environment

6.2. Information Flow

The information flow captures the dynamic execution of the testing process, from initial level extraction to the final generation of failure reports.

6.2.1. System Activity Diagram

The activity diagram outlines the high-level logic of the automated testing workflow. It specifically highlights the decision loop where the system validates moves against the 7-slot buffer constraint and checks for ”soft-lock” failure conditions.

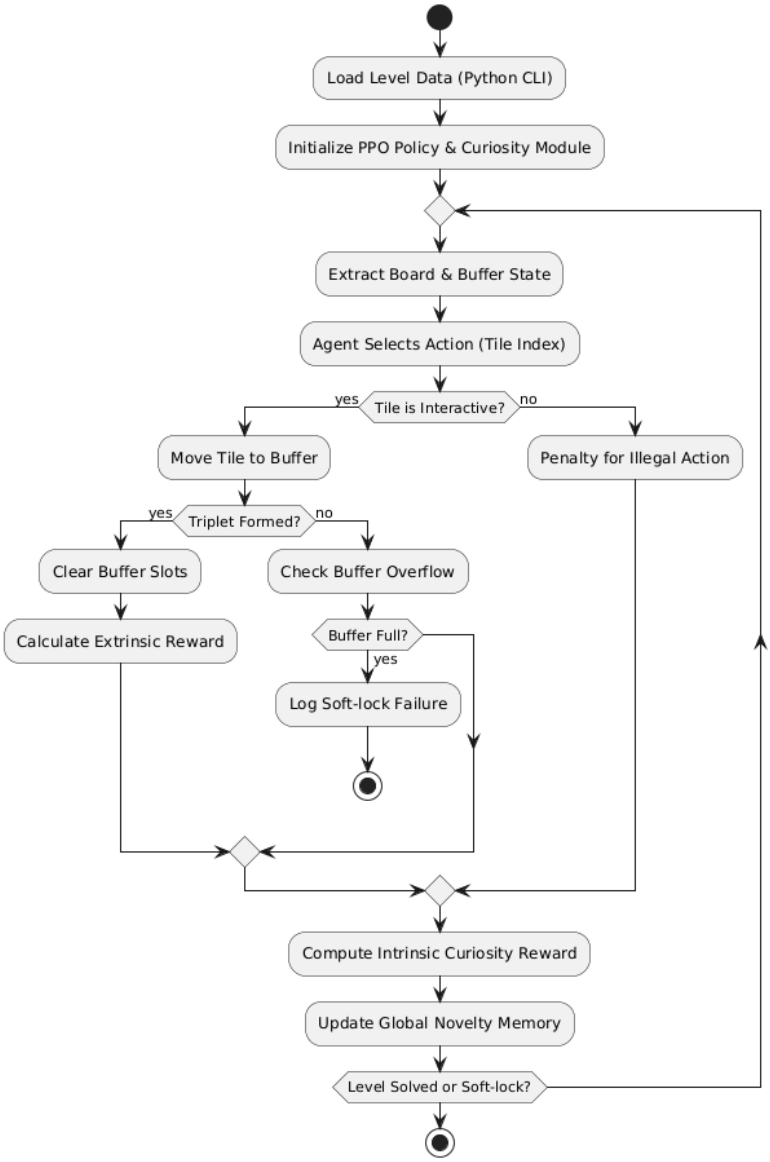

Figure 6.2. System Activity Diagram for automated RL-based game testing

6.2.2. Agent-Environment Sequence Diagram

This diagram illustrates the message passing between the PPO Agent, the Python-based engine, and the Curiosity Module. It details how intrinsic rewards are calculated based on state-transition novelty and how buffer-overflow signals terminate an exploration episode.

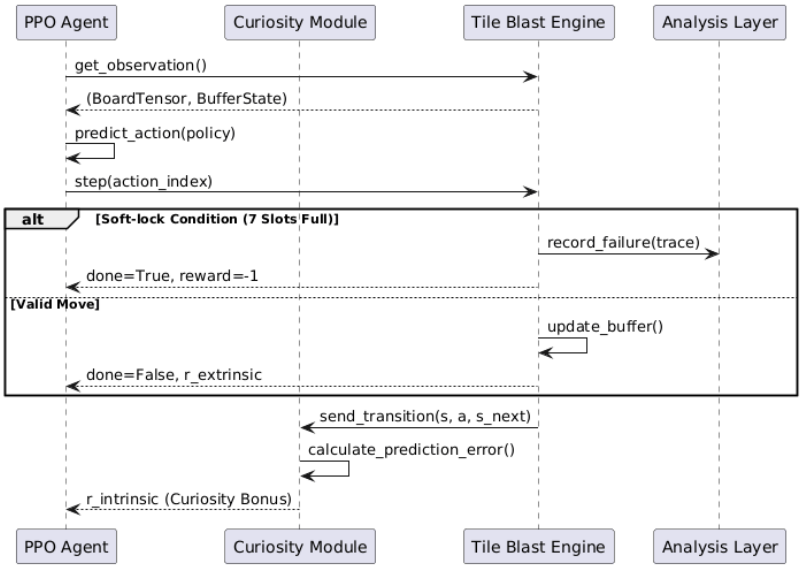

Figure 6.3. Agent-Environment Sequence Diagram showing PPO and ICM interactions

6.3. System Design

The system design follows a modular approach to maintain a strict separation between the game engine logic and the reinforcement learning pipeline.

6.3.1. Class Diagram

The class diagram shows the primary objects within the repository. Key classes include `TileBlastEnv`, which acts as the primary interface, and `BufferManager`, which encapsulates the triple-match logic and capacity constraints.

6.3.2. Module Diagram

The framework is decomposed into three primary modules to facilitate high-throughput execution:

- **engine_core:** Contains the lightweight Python CLI logic for *Tile Blast* and *Wiggle Escape*.
- **rl_pipeline:** Encompasses the PPO agent, the Intrinsic Curiosity Module (ICM),

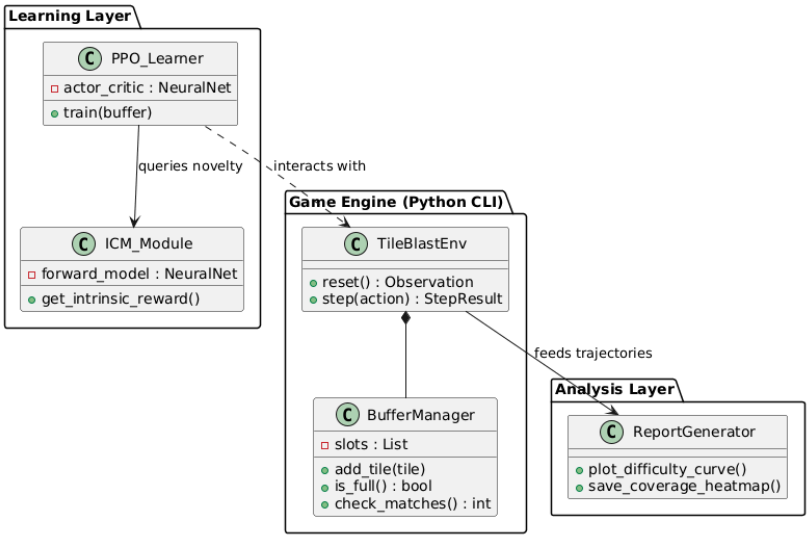

Figure 6.4. Class Diagram of the curiosity-driven testing framework

and player profile modifiers.

- **analysis_suite:** Responsible for transforming raw agent trajectories into designer artifacts such as coverage heatmaps and difficulty curves.

6.4. User Interface Design

As the framework is optimized for automated high-speed simulation, the user interface is primarily a Terminal-based CLI.

- **Debug View:** A colorized ANSI grid representing the tile stacks and the current 7-slot buffer status, used for verifying agent behavior.
- **Analytical Reports:** Post-process visualizations generated by the analysis suite to provide designers with clear indicators of level stability and difficulty spikes.

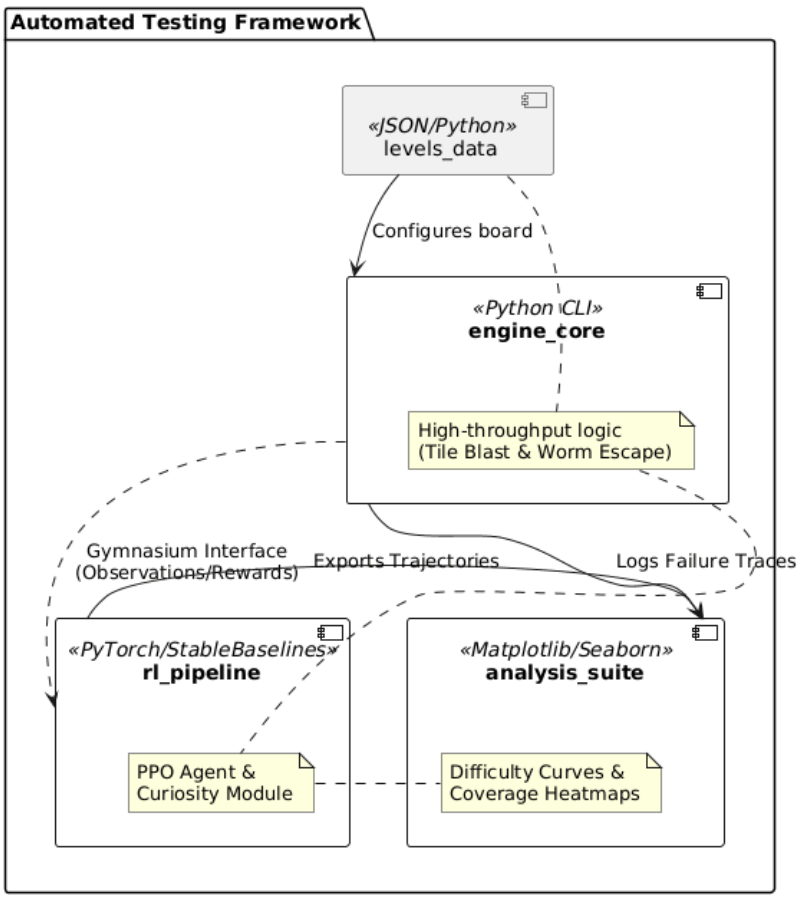

Figure 6.5. Module Diagram showing high-level framework decomposition

7. IMPLEMENTATION AND TESTING

This chapter describes the current implementation of the curiosity-driven testing framework, focusing on the lightweight puzzle engine, diagnostic tools, and deployment considerations for high-throughput simulation.

7.1. Implementation

The current implementation establishes a Python-based command-line interface (CLI) environment optimized for rapid simulation. All components operate on a shared `LEVEL_DATA` contract, ensuring consistency across the engine, solver, and procedural generator.

7.1.1. Worm Escape Engine Implementation

The Worm Escape engine served as the initial testbed and is fully implemented with the following components:

Core Logic: The `engine.py` and `entities.py` modules manage spatial occupancy and movement mechanics, supporting directional extraction puzzles.

Level Management: The `level_manager.py` module handles grid parsing, metadata validation, and structural constraints, ensuring that worm heads are positioned correctly according to their declared direction.

Interactive Tools: A CLI-based Level Editor supports manual level authoring with real-time validation. Additionally, a Procedural Generator creates guaranteed-solvable levels via reverse dependency construction.

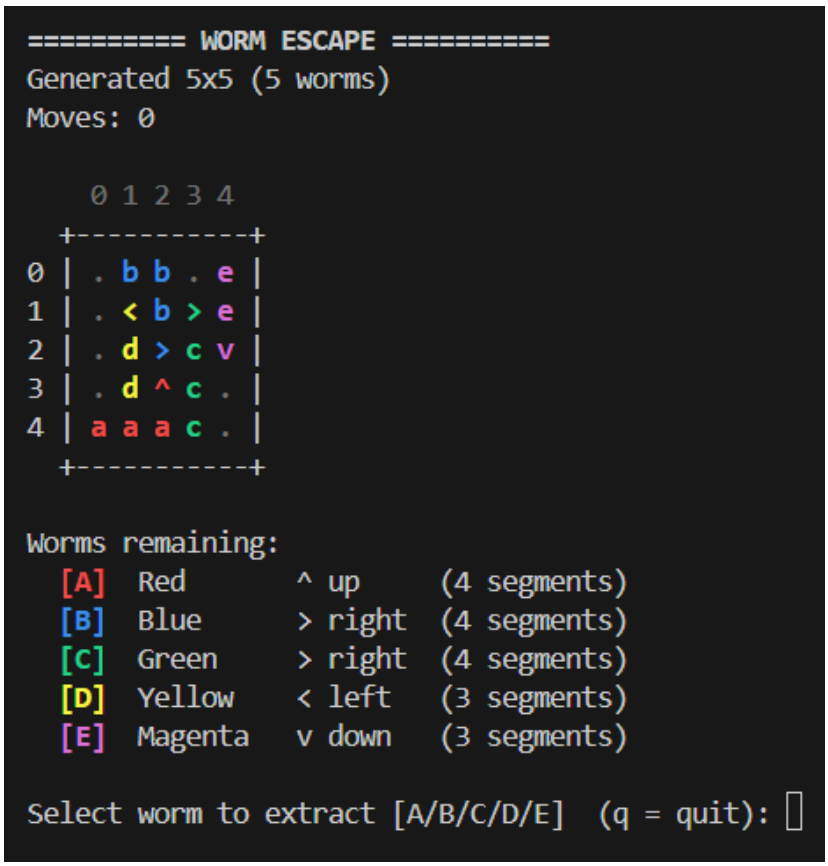

Figure 7.1. Example screenshot of the Worm Escape engine in CLI.

7.1.2. Transition to Tile Blast

Development has transitioned to the Tile Blast environment while maintaining the lightweight CLI philosophy:

- **Layered State Logic:** Implements 3D stacking, visibility, and interaction rules for tiles.
- **Buffer Management:** Enforces the strict 7-slot tray constraint and triggers automatic triplet-clearing logic.

7.2. Testing

Testing focuses on logical validation and level solvability, ensuring the quality of environments prior to RL agent training.

7.2.1. Graph-Based Solvability Checker

A key tool is the Automated Solver (`solver.py`), which validates levels using directed dependency graphs:

- **Dependency Mapping:** Constructs a "blocked-by" graph for each worm by tracing interactions with other entities.
- **Iterative Extraction:** Simulates sequential extraction of worms with zero dependencies until the level is solved or a deadlock occurs.
- **Explainable Reporting:** Produces a round-by-round report detailing extraction logic and deadlock reasoning, enhancing interpretability for designers.

7.2.2. Procedural Generator Validation

The procedural generator is validated by cross-referencing generated levels with the graph-based solver. Only levels successfully solved by the solver are used for RL agent training, ensuring reproducibility and data integrity.

7.3. Deployment

The framework is designed for local development and high-throughput simulation, with future plans for containerized deployment.

Environment Requirements: Python 3.11+; ANSI escape codes for terminal-based rendering.

Build Instructions: Launch `main.py` to initialize the game loop, load levels, and start the simulation.

Future Deployment: The RL training pipeline will be containerized using Docker to ensure deterministic execution across different compute environments. This will facilitate reproducible experiments and simplified scaling for larger agent swarms.

8. RESULTS

As this is a midterm report, the results presented here are preliminary and primarily document the validation of the foundational puzzle engine and deterministic solver. The implementation of curiosity-driven reinforcement learning (RL) agents is currently in progress.

8.1. Worm Escape Validation

The initial phase of the project focused on developing and verifying the Worm Escape environment to test the high-throughput capabilities of the Python CLI architecture.

Deterministic Solver Accuracy: The graph-based dependency solver successfully validated 100% of the hand-authored tutorial levels. It correctly identified extraction orders and provided detailed reasoning for detected deadlocks, confirming the reliability of the underlying game logic.

Procedural Generation Consistency: The reverse-construction algorithm in the procedural generator produced levels that were 100% solvable according to the independent solver module. This ensures that valid training data for future RL agents can be generated reliably.

Simulation Performance: Benchmark tests of the Python CLI indicate that state transitions and solver logic execute in under 10 ms per level, meeting the high-throughput non-functional requirement for large-scale RL training.

8.2. Tile Blast Implementation Status

The transition to the Tile Blast environment is currently at an early implementation stage. Consequently, RL training results and performance metrics are not yet

available.

Current Progress: The structural design and state-extraction logic for the Mahjong-style 3D stacks and the 7-slot buffer have been finalized.

Pending Implementation: The core gameplay loop and match-3 mechanics for the CLI version of Tile Blast are under development to satisfy the architectural requirements defined in Chapter 6.

Expected Outcomes: Once the environment stabilizes, it will be used to benchmark the state-space coverage of curiosity-driven agents against the baseline defined by the manual solver.

8.3. Preliminary Research Findings

Initial research conducted during environment modeling has led to several observations:

Dependency Complexity: Analysis of the Worm Escape solver indicates that while directed dependency graphs suffice for 2D extraction puzzles, they do not scale effectively to the resource-management constraints present in Tile Blast.

The Buffer Constraint: Theoretical modeling suggests that the 7-slot buffer in Tile Blast will act as the primary bottleneck for agent progression. Intrinsic curiosity will be necessary to prevent early episode termination caused by buffer "clogging".

9. CONCLUSION

This project represents a significant step toward bridging the gap between advanced Reinforcement Learning (RL) research and practical, high-speed game testing workflows. By moving away from heavy graphical engines toward a high-throughput Python-based CLI, we have established a framework capable of exhaustive state-space exploration at a fraction of the traditional computational cost.

9.1. Summary of Progress

To date, the project has achieved several critical milestones:

- **Theoretical Foundation:** A comprehensive literature review was conducted, situating the project within the context of curiosity-driven exploration and multi-agent coordination.
- **Environment Architecture:** The "Three-Layer Architecture" (Learning, Coordination, and Analysis) has been fully designed to ensure a modular separation between game logic and agent training.
- **Worm Escape Implementation:** A complete ecosystem for directional extraction puzzles was developed, including a game engine, an interactive level editor, and a procedural generator.
- **Deterministic Solver:** A graph-based dependency solver was implemented and verified, providing a 100% accurate baseline for level solvability and explainable failure reporting.
- **System Design for Tile Blast:** The architectural blueprints for the primary Tile Blast testbed—including the complex 3D-stacking logic and the 7-slot buffer constraints—have been finalized in the form of UML and ER diagrams.

9.2. Challenges and Insights

The development of the Worm Escape solver revealed that while deterministic algorithms are highly efficient for path-based puzzles, they lack the flexibility required for resource-management games like Tile Blast. This observation underscores the necessity of the proposed curiosity-driven RL approach, which is better suited for navigating "clogging" scenarios and discovering non-intuitive edge cases within limited-capacity buffers.

9.3. Future Work

The second half of the project will focus on the full-scale implementation and evaluation phase:

- **Tile Blast CLI Completion:** Finalizing the Python-based match-3 logic and 3D visibility layers.
- **RL Integration:** Deploying the Proximal Policy Optimization (PPO) agent and the Intrinsic Curiosity Module (ICM) to initiate the learning phase.
- **Swarm Coordination:** Implementing the global novelty memory to synchronize parallel agents and maximize state-space coverage.
- **Analytical Evaluation:** Generating the final "Designer Artifacts," including difficulty curves and heatmaps, to validate the framework's utility in a real-world Live-Ops environment.

The work completed thus far provides a robust, engineered foundation. Upon completion of the Tile Blast learning layer, the framework will offer game designers a scalable, automated assistant capable of quantifying difficulty and identifying logic errors with minimal manual intervention.

REFERENCES

1. Ferdous, H., B. Marin, A. Iglesias and T. Vos, "Curiosity-Driven Reinforcement Learning for 3D Game Testing", *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 1142–1153, 2022.

2. Ferdous, H., T. Vos, B. Marin and A. Iglesias, "Curiosity-Driven Multi-Agent Reinforcement Learning for 3D Game Testing", *arXiv preprint arXiv:2502.14606*, 2025.

3. Bergdahl, J., C. Gordillo, K. Tollmar and L. Gisslén, "Augmenting Automated Game Testing with Deep Reinforcement Learning", *IEEE Conference on Games (CoG)*, pp. 497–504, 2020.

4. Zheng, Y., X. Xie, T. Su, L. Ma, J. Hao *et al.*, "Wuji: Automatic Online Combat Game Testing using Evolutionary Deep Reinforcement Learning", *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 772–784, 2019.

APPENDIX A: LEVEL DATA SPECIFICATION

This appendix details the standardized `LEVEL_DATA` dictionary format used to define puzzle states within the *Worm Escape* ecosystem. This schema is shared across the gameplay engine, the graph-based solver, and the procedural generator to ensure architectural consistency.

A.1. Schema Definition

The level configuration utilizes a Python dictionary with the following primary keys:

- **name:** A descriptive string identifying the level for the user interface.
- **rows / cols:** Integer values defining the dimensions of the grid.
- **grid:** A list of strings where each character represents a cell in the puzzle.
  - Uppercase Letters (A-Z): Represent the head of a specific worm.
  - Lowercase Letters (a-z): Represent the body segments belonging to the corresponding uppercase head.
  - Period (`.`): Represents an empty, traversable cell.
- **worms:** A nested dictionary mapping each WormID (e.g., "A") to its specific movement and aesthetic properties.
  - **color:** Defines the ANSI color rendering used in the CLI.
  - **direction:** The fixed extraction path (`up`, `down`, `left`, `right`) for that specific entity.

A.2. Representative Implementation

The following code block demonstrates the specification for a complex dependency-based level:

```
LEVEL_DATA = {
    "name": "Level 1 | First Steps",
    "rows": 6,
    "cols": 6,
    "grid": [
        "......",
        "...Cc.", # Head 'C' at (1,3), body 'c' at (1,4)
        "...B..", # Head 'B' at (2,3)
        "aaAb..", # Head 'A' at (3,2), body 'b' at (3,3)
        "...b..", # Body 'b' at (4,3)
        "......",
    ],
    "worms": {
        "A": {"color": "red",   "direction": "right"},
        "B": {"color": "blue",  "direction": "up"},
        "C": {"color": "green", "direction": "left"},
    },
}
```

A.3. Structural Validation Rules

The `level_manager.py` module enforces the following constraints upon loading:

- **Head Uniqueness:** Each WormID must have exactly one uppercase head character in the grid.
- **Leading-Edge Rule:** For the declared direction, the head must be the segment closest to the extraction point (e.g., if direction is "up", no body segment can exist at a lower row index than the head).
- **Segment Connectivity:** All body segments must form a contiguous, non-branching path connected to the head.

APPENDIX B: SOLVER DIAGNOSTIC REPORT

To demonstrate the "Explainable Reporting" capability of the framework, this appendix provides a step-by-step diagnostic trace for **Level 1 — First Steps**.

B.1. Level Data Configuration

The following dictionary defines the spatial and directional constraints of the test environment:

```
LEVEL_DATA = {
    "name": "Level 1 | First Steps",
    "rows": 6,
    "cols": 6,
    "grid": [
        "......",
        "...Cc.", # C (Head) at [1,3], c (Body) at [1,4]
        "...B..", # B (Head) at [2,3]
        "aaAb..", # A (Head) at [3,2], b (Body) at [3,3]
        "...b..", # b (Body) at [4,3]
        "......",
    ],
    "worms": {
        "A": {"color": "red",   "direction": "right"},
        "B": {"color": "blue",  "direction": "up"},
        "C": {"color": "green", "direction": "left"},
    },
}
```

B.2. Dependency Analysis Trace

The `solver.py` module executes the following logic to determine solvability:

B.2.1. Initial Dependency Mapping

- **Worm A (Right):** Blocked at coordinate [3,3] by Worm B.
- **Worm B (Up):** Blocked at coordinate [1,3] by Worm C.
- **Worm C (Left):** Has no occupancy in its path (Columns 0–2 of Row 1 are empty).

B.2.2. Extraction Round 1

- Worm C is identified with zero dependencies and successfully extracted from the grid.
- The "Blocked-By" graph is updated; the occupancy at [1,3] is cleared.

B.2.3. Extraction Round 2

- Worm B's path is now clear. The engine executes the extraction, removing segments at [2,3], [3,3], and [4,3].
- The occupancy at [3,3] is cleared, releasing the dependency for Worm A.

B.2.4. Extraction Round 3

- Worm A is extracted successfully.

B.2.5. Final Diagnostic Result

- **Solvability:** True
- **Optimal Sequence:** C $\rightarrow$ B $\rightarrow$ A
- **State-Space Coverage:** 100% of required moves executed without deadlock